### 10.2.2 Linking components to scales

To do that, we have to write a new function for our legend that takes a scale and derives the necessary range bands to be useful. The scale that we send it will be the same `countryColor` scale that we use to color our grid circles. Because this is a quantize scale, we'll make our legend component hardcoded to handle only quantize scales. If we wanted to make this a more robust component, we'd need to make it identify and handle the various scales that D3 uses.

Just like all scales have an invert function, they also have the ability to tell you what domain values are mapped to what range values. First, we need to know the range of values of our quantize scale as they appear to the scale. We can easily get that range by using `scale.quantize.range()`:

```
countryColor.range()
```

**["#fee5d9", "#fcbba1", "#fc9272", "#fb6a4a", "#ef3b2c", "#cb181d", "#99000d"]**

We can pass those values to `scale.quantize.invertExtent` to get the numerical domain mapped to each color value:

```
countryColor.invertExtent("#fee5d9")
```

**[0.00000674650100275 9535, 0.05946855349777645]**

Armed with these two functions, all we need to do now is give our legend component the capacity to have a scale assigned to it and then update the legend function itself to derive from that scale the dataset necessary for our legend. Listing 10.13 shows both the new `d3.legend.scale()` function that uses a quantize scale to create the necessary dataset, and the updated `legend()` function that uses that data to draw a more meaningful set of `<rect>` elements.

**Listing 10.13 Updated `legend` function**

```
d3.svg.legend = function() {
       var data = [];
       var size = [300,20];
       var xScale = d3.scale.linear();
       var scale;

       function legend(gSelection) {

       createLegendData(scale);

       var xMin = d3.min(data, function(d) {return d.domain[0];});
       var xMax = d3.max(data, function(d) {return d.domain[1];});
   xScale.domain([xMin,xMax]).range([0,size[0]])

   gSelection.selectAll("rect")
      .data(data)
      .enter()
      .append("rect")
      .attr("height", size[1])
```

**Sets a default size**

**Initializes an x-axis scale but doesn't set domain or range**

**The scale that will be sent to the component**

**Calls the function to process the scale into a data array**

**Calculates the min/max of the scale data**

**Sets the x-axis scale**

**Draws rectangles based on component settings and scale data**